Course Module: Deploy you AI model

Latreche Sara

Contents

1 Course Overview **2**

2 Introduction to Flask **2**

2.1 HTTP Verbs 4

2.2 Serving Machine Learning Models 7

3 From Flask to FastAPI **7**

4 Why Docker? Intuition First **10**

4.1 The Dockerfile 10

4.2 From One Container to Many: Docker Compose 11

4.3 Best Practices for Building Images 12

4.4 Docker for Data Scientists and AI 12

5 Kubernetes (K8s) **13**

6 CI/CD: Continuous Integration and Continuous Deployment **15**

7 MLOps and Model Management **18**

8 Project Structure & Best Practices **19**

1 Course Overview

Have you ever wondered how apps like YouTube or Instagram use AI to recommend videos or posts tailored to you? How can you take the models you explore in Jupyter notebooks and turn them into interactive applications for real users?

In this final stage of the Data Science Bootcamp, you will build your AI project from end to end. You will learn how to structure your project efficiently, share it with others, and turn it into a usable product. Jupyter notebooks are great for experimentation, but here you will transform your exploratory code into deployable applications using tools like Flask, REST APIs, and containerization with Docker.

By the end of this course, you will be able to:

- Learn how to expose AI/ML models using Flask.
- Understand containerization with Docker.
- Practice deploying models using REST APIs.
- Build and deploy applications locally and in the cloud.

2 Introduction to Flask

Definition

Flask is a lightweight Python web framework that allows you to build web apps and APIs quickly and flexibly.

Why Flask?

Flask is especially useful in AI and machine learning because it enables you to expose your model via a REST API — so others (or other systems) can send inputs and get predictions over the web.

- **Minimalist**: Flask provides only what you need.
- **Modular**: You can extend it with plugins or custom code.
- **Flexible**: No fixed folder structure or ORM.

Note

Alternatives include Django (full-stack framework) and FastAPI (modern, async, and high-performance).

Core Components of Flask

- **WSGI (Web Server Gateway Interface)**: A specification that defines how a web server communicates with a Python application.

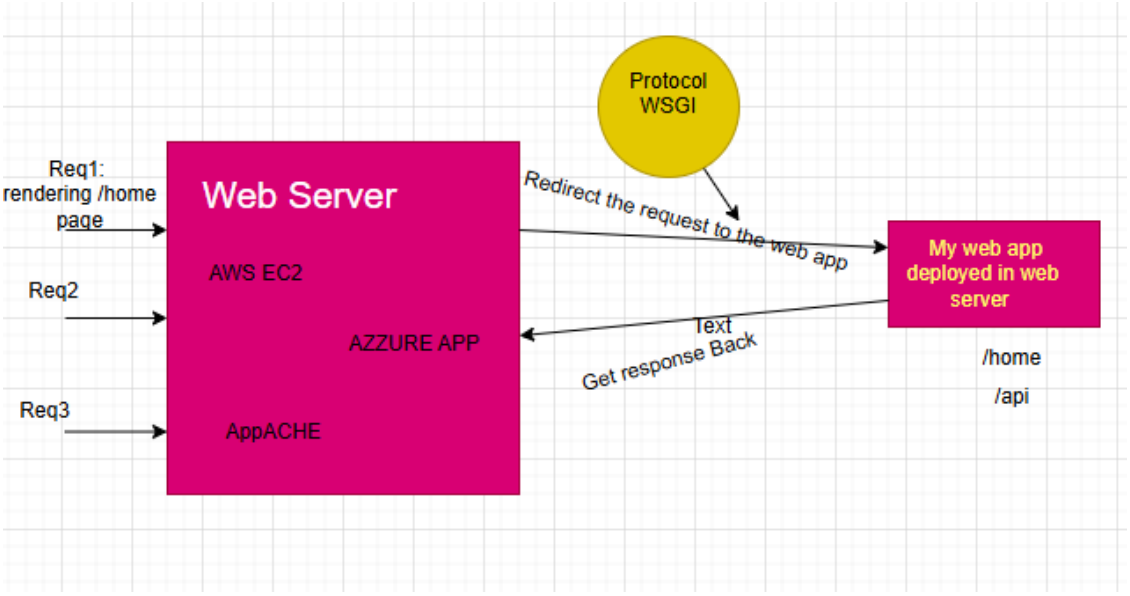


Figure 1: **Understanding WSGI:** The web server receives a request and uses the WSGI protocol to communicate with your Flask web app, which processes the request and returns a response.

- **Jinja2 Template Engine**: Used for rendering dynamic HTML templates from Python variables.

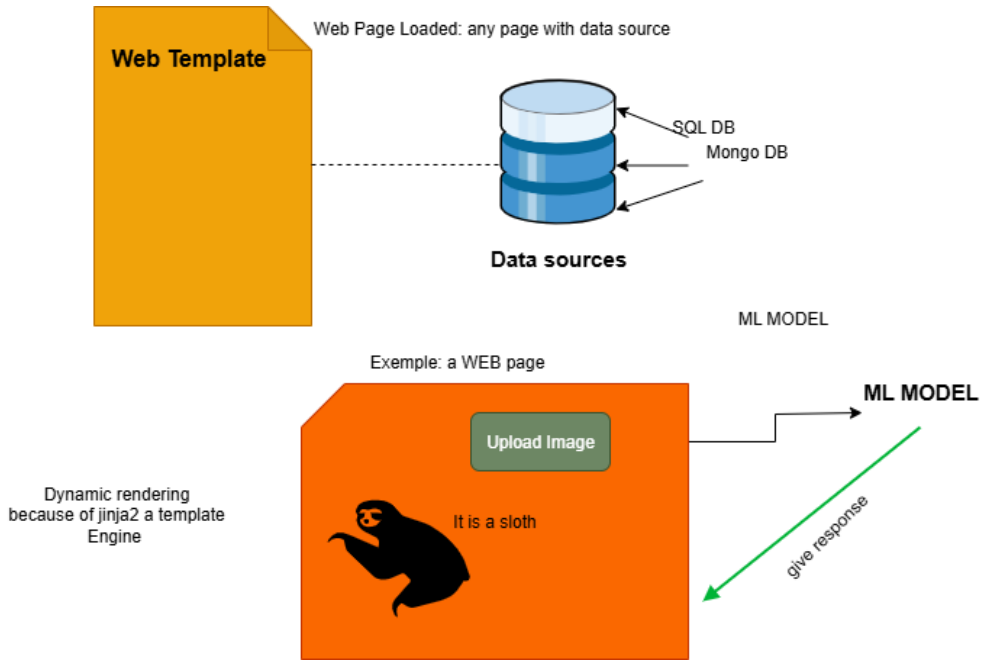


Figure 2: **Jinja2 Template Engine:** Jinja2 allows Flask to render dynamic content on web pages by connecting to various data sources or ML models. For example, a user uploads an image, the model processes it, and Jinja2 renders the result dynamically on the webpage.

Active Recall – Recall Questions

1. What is **Flask**, and what are some alternative frameworks to it?
2. What is the purpose of a web framework like Flask in the context of machine learning or app development?
3. Flask has two major components that were mentioned. What are they?
4. What is the role of the **Web Server Gateway Interface (WSGI)** in Flask?
5. What does **Jinja2** do in Flask?
6. Can you list examples of **data sources** Flask can connect to?
7. What protocol/interface does the web server use to talk to your Flask app?

2.1 HTTP Verbs

HTTP defines a set of request methods to indicate the desired action to be performed for a given resource. These methods are also called HTTP verbs. The most commonly used HTTP verbs are:

HTTP Verbs

- **GET** – Retrieves data from the server (e.g., reading a webpage).
- **POST** – Sends data to the server, typically used for creating new resources.
- **PUT** – Updates an existing resource or creates a new one if it doesn't exist.
- **DELETE** – Deletes a specified resource.
- **PATCH** – Partially updates an existing resource.
- **HEAD** – Retrieves headers only, without the response body.
- **OPTIONS** – Describes the communication options for the target resource.

Understanding these verbs is crucial for designing RESTful APIs and interacting with web services.

Test Your Knowledge

Try to answer each question **without looking**. Then, check your answers below.

Questions

1. What HTTP method is used to retrieve data from a server without modifying it?
2. Which method is used to send data to the server to create a new resource?
3. What is the difference between `PUT` and `PATCH`?
4. Which method is considered idempotent: `POST`, `PUT`, or `DELETE`?
5. What does the `DELETE` method do?
6. Which method is commonly used in HTML forms?
7. What is the purpose of the `OPTIONS` method?
8. What is the `HEAD` method used for?

Exercise: Building and Modifying a RESTful API with Flask

Exercise: PUT and DELETE Methods

In this exercise, you'll build a mini RESTful API using Flask to manage a to-do list. You'll work with various HTTP methods and JSON responses.

Instructions

1. Create a Flask application that manages a to-do list of items. Each item should be a dictionary with `id`, `name`, and `description`.
2. Implement the following routes and methods:
   - `GET /items` – Returns all items.
   - `GET /items/<id>` – Returns a specific item by ID.
   - `POST /items` – Adds a new item (from JSON payload).
   - `PUT /items/<id>` – Updates an existing item.
   - `DELETE /items/<id>` – Deletes an item.
3. For updating (`PUT`), ensure partial updates work (keep old data if not included).
4. For deleting, update the in-memory list.
5. Handle edge cases where the item is not found (e.g. return `"error": "Item not found"`).

Questions

- What HTTP method is typically used to completely update a resource?
- What is the difference between `PUT` and `PATCH`?
- How do you retrieve request data from a client in Flask?
- Why is JSON used in API responses?
- What should a DELETE request return?

Test your solution: Run the server locally and use a tool like `curl`, Postman, or a browser to make requests.

2.2 Serving Machine Learning Models

You can load a trained machine learning model inside a Flask app and expose it through an API, turning your exploratory Jupyter notebooks into interactive applications.

Example

Suppose you trained a Scikit-learn model to predict housing prices. You can save it using `joblib`, then write a Flask route like `/predict` that accepts JSON input and returns predictions.

Steps Overview

1. Train your model (e.g., using Scikit-learn, TensorFlow, or PyTorch).
2. Save the model using `joblib` or `pickle`.
3. Load the model inside your Flask app.
4. Write endpoints to receive requests and return predictions.

Definition

REST API: A standardized way to expose resources like ML predictions via HTTP verbs such as GET and POST.

Note

Flask is perfect for prototyping, but in production, you should serve your app with a WSGI server like `gunicorn` for stability and performance.

3 From Flask to FastAPI

Definition

FastAPI is a modern, high-performance Python web framework built on ASGI (Asynchronous Server Gateway Interface). It is designed for building APIs quickly with automatic interactive documentation.

Why Move from Flask to FastAPI?

While Flask is excellent for learning and rapid prototyping, FastAPI is particularly useful when your applications need to handle high traffic or require automatic validation:

- **Speed**: Built on `Starlette` and `Pydantic`, FastAPI is significantly faster than Flask.
- **Automatic Documentation**: Endpoints generate live interactive docs at `/docs`.
- **Async Support**: Natively supports asynchronous endpoints for high-performance applications.
- **Validation**: Input data is automatically validated against type hints.

Note

Think of it this way: **Flask** is like a flexible notebook script for sharing your model. **FastAPI** is like a production-ready engine that also generates a manual for your API automatically!

Quick Comparison

	Flask	**FastAPI**
Protocol	WSGI	ASGI (async-ready)
Docs	Manual (Swagger via extensions)	Automatic interactive docs (`/docs`)
Performance	Good for small/medium apps	Very high (near Node.js / Go)
Validation	Manual (e.g., Marshmallow)	Automatic from Python type hints

Example: Same API in Flask vs FastAPI

Example

Flask:

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route("/hello", methods=["GET"])
def hello():
    return jsonify({"message": "Hello, world!"})
```

FastAPI:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/hello")
async def hello():
    return {"message": "Hello, world!"}
```

Hands-on Lab: First FastAPI App

Hands-on Lab

Your First FastAPI App

1. Install FastAPI and Uvicorn:

   ```
   pip install fastapi uvicorn
   ```

2. Create `main.py` with the FastAPI example above.

3. Run the app:

   ```
   uvicorn main:app --reload
   ```

4. Visit `http://127.0.0.1:8000/hello` and test.

5. Explore auto-generated docs at `/docs`.

4 Why Docker? Intuition First

Definition

Docker is a platform that allows you to package, distribute, and run applications inside *containers* — lightweight, self-contained environments that bundle code with all its dependencies.

Docker vs Virtual Machines (VMs)

- **VMs**: Heavy, each has its own OS kernel.
- **Docker**: Lightweight, containers share the host OS kernel.
- **Result**: Faster startup, smaller size, easier sharing.

—

4.1 The Dockerfile

Definition

A **Dockerfile** is a recipe that describes how to build a Docker image.

General Structure

```
FROM <base-image>
RUN <command-to-install-dependencies>
COPY <local-path> <container-path>
WORKDIR <container-path>
CMD ["executable", "param1", "param2"]
```

—

Hands-on Lab

Lab 1: Build Your First Image

1. Create a file `Dockerfile` with:

```
FROM ubuntu:20.04
RUN apt-get update && apt-get install -y python3
CMD ["python3", "--version"]
```

2. Build the image:

```
docker build -t mypython .
```

3. Run it:

```
docker run mypython
```

- What is the difference between an `image` and a `container`? - Why do we use `CMD` instead of `RUN` for commands?

—

4.2 From One Container to Many: Docker Compose

Definition

Docker Compose is a tool for defining and running multi-container applications (e.g., app + database + cache) using a single YAML file.

Intuition

Imagine you are a data scientist:

- One container runs your Jupyter Notebook.
- Another container runs PostgreSQL.
- A third container runs Redis cache.

Instead of starting each manually, you orchestrate them together with `docker-compose.yml`.

—

Hands-on Lab

Lab 2: Compose Your Services

1. Create `docker-compose.yml`:

```
version: "3.8"
services:
  web:
    build: .
    ports:
      - "5000:5000"
  db:
    image: postgres:13
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
```

2. Start all services:

```
docker-compose up
```

- Why does Compose make collaboration easier? - What happens to your database data if you stop the container?

—

4.3 Best Practices for Building Images

- Use small base images (e.g., `python:3.10-slim`).
- Add a `.dockerignore` file to skip unnecessary files.
- Pin versions of dependencies.
- Keep layers minimal (combine multiple `RUN` commands when possible).

—

4.4 Docker for Data Scientists and AI

Advantages

- Reproducible experiments.

- Easy sharing of models and environments.
- Compatible with cloud deployments.

Example: Hugging Face Transformers

Hands-on Lab

Lab 3: Run a Hugging Face Model in Docker Dockerfile:

```
FROM python:3.10-slim
RUN pip install transformers torch flask
COPY app.py /app/app.py
WORKDIR /app
CMD ["python", "app.py"]
```

Run the container:

```
docker build -t hf-agent .
docker run -p 8080:8080 hf-agent
```

- Why is Docker essential for deploying AI agents in production? - How does it help with dependency conflicts (e.g., TensorFlow vs PyTorch)?

5 Kubernetes (K8s)

Why Kubernetes?

Docker Compose is excellent for local development and small apps, but real-world systems need more advanced features:

- **Scaling**: run many copies of a service across multiple machines.
- **Self-healing**: automatically restart failed containers.
- **Load balancing**: distribute traffic evenly across replicas.
- **Cluster orchestration**: manage containers across several servers.

Definition

Kubernetes (K8s) is a container orchestration system that automates deployment, scaling, networking, and management of containerized applications.

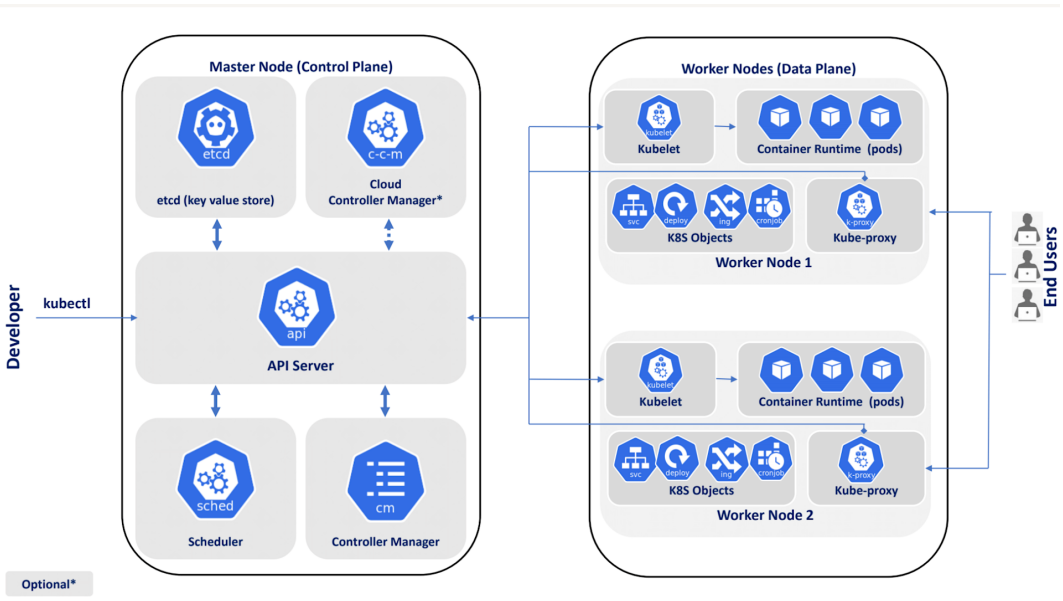


Figure 3: **Basic Kubernetes cluster architecture:** Pods, Nodes, and Control Plane.

Kubernetes vs Docker Compose

Key Differences

- **Docker Compose**: single-node, local development, YAML-defined services.
- **Kubernetes**: multi-node, production-grade orchestration with scaling, resilience, and self-healing.

Note

Think of Docker Compose as a "sandbox" for testing multi-container setups. Kubernetes is the "factory" that runs them at scale in production.

Core Kubernetes Concepts

- **Pod** – smallest deployable unit; can contain one or more containers.
- **Service** – provides a stable network endpoint for accessing Pods.
- **Deployment** – declarative management of Pods and replicas.
- **ConfigMap / Secret** – store configuration data and sensitive information securely.

Example

A Deployment defines the "desired state," e.g., 3 replicas of your Flask API. Kubernetes continuously ensures this state is met, restarting Pods if they fail.

Hands-on Lab: Deploying Flask on Kubernetes

Hands-on Lab

Deploy Your Flask API with Minikube

1. Install Minikube and kubectl.
2. Write a `Deployment.yaml` for your Flask app.
3. Write a `Service.yaml` to expose the app.
4. Run:

```
kubectl apply -f Deployment.yaml
kubectl apply -f Service.yaml
```

5. Access your API through Minikube's service URL.

- Why does Kubernetes need both a `Deployment` and a `Service`? - How does Kubernetes ensure "self-healing" of Pods? - What are the advantages of using Kubernetes over Docker Compose in production environments?

6 CI/CD: Continuous Integration and Continuous Deployment

Definition

CI/CD stands for **Continuous Integration** and **Continuous Deployment (or Delivery)**. It is a set of practices and tools to automate the process of building, testing, and deploying software.

Why CI/CD?

- **Faster Time to Market:** Automate builds and deployments to release features quickly.

- **Improved Code Quality:** Automated tests catch errors early.
- **Reduced Risk:** Small, frequent updates are easier to test and roll back if needed.
- **Enhanced Collaboration:** Multiple developers can work simultaneously without conflicts.

Typical CI/CD Workflow

1. Developer pushes code to a repository (GitHub, GitLab, etc.).
2. CI server (e.g., GitHub Actions, Jenkins) builds the code.
3. Automated tests run to verify correctness.
4. If tests pass, the image or artifact is pushed to a registry (Docker Hub, etc.).
5. Deployment pipeline deploys it automatically to staging or production.

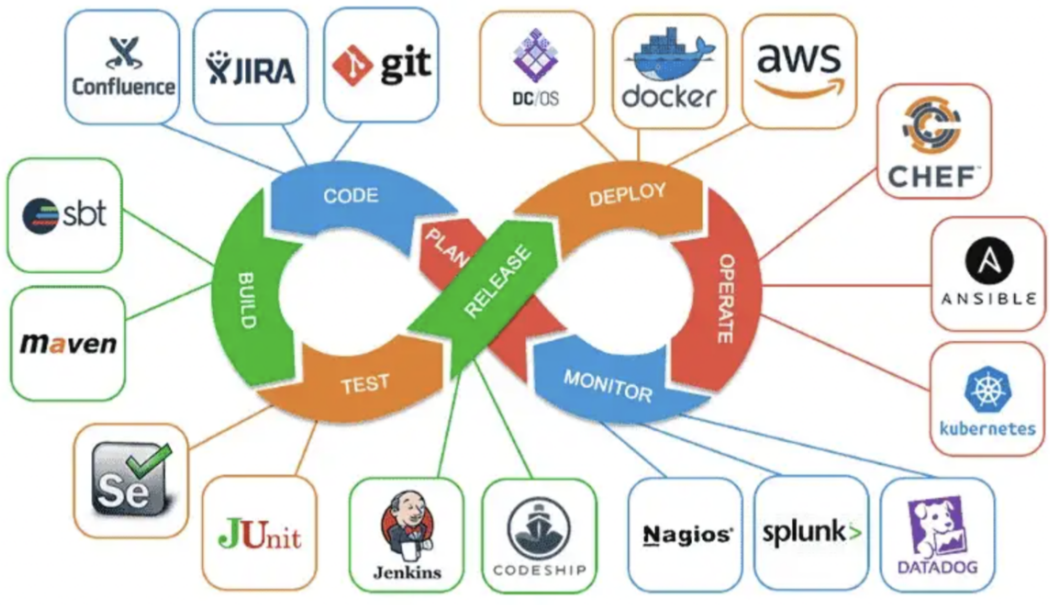


Figure 4: **CI/CD Pipeline:** From code commit to automatic deployment.

Key Tools for CI/CD

Examples of Tools

- **Version Control:** Git, GitHub, GitLab
- **CI/CD Platforms:** GitHub Actions, Jenkins, GitLab CI, CircleCI
- **Build Tools:** Maven, Gradle, npm
- **Testing Frameworks:** JUnit, PyTest, Selenium
- **Deployment Tools:** Docker, Kubernetes, AWS CodePipeline

Note

Think of CI/CD as a conveyor belt: code goes in, and tested, deployable software comes out automatically. It reduces manual errors and saves time.

Lab/Exercise Idea

Hands-on Lab

Simple CI/CD with GitHub Actions

1. Create a simple Python project and push it to GitHub.
2. Write a `.github/workflows/ci.yml` workflow that:
   - Installs dependencies
   - Runs tests
   - Builds a Docker image
3. Trigger the workflow by pushing a change.
4. Observe the automated build and test results in GitHub Actions.

7 MLOps and Model Management

Definition

MLOps (Machine Learning Operations) is a set of practices that combines Machine Learning, DevOps, and Data Engineering to automate and streamline the deployment, monitoring, and management of ML models in production.

Why MLOps Matters

- Ensures **reproducibility** of experiments.
- Automates **training, testing, and deployment** pipelines.
- Monitors model performance in production to detect **drift or degradation**.
- Makes collaboration easier between **data scientists, engineers, and DevOps**.

Definition

MLflow is an open-source platform to manage the ML lifecycle, including experimentation, reproducibility, deployment, and monitoring.

MLflow Components

- **MLflow Tracking:** Log and query experiments and metrics.
- **MLflow Projects:** Package code in a reusable and reproducible way.
- **MLflow Models:** Manage and deploy models across multiple platforms.
- **MLflow Registry:** Central store to version and manage model lifecycle.

Hands-on Lab

Hands-On Lab: Tracking Experiments with MLflow

1. Install MLflow with `pip install mlflow`.
2. Train a simple model (e.g., linear regression with scikit-learn).
3. Log parameters, metrics, and the model using Python code:

```
import mlflow
import mlflow.sklearn

with mlflow.start_run():
    mlflow.log_param("alpha", 0.5)
    mlflow.log_metric("rmse", 1.23)
    mlflow.sklearn.log_model(model, "linear_model")
```

4. Start MLflow UI with `mlflow ui` and visit `http://127.0.0.1:5000` to explore runs.

Integration with CI/CD and Docker

- Track ML experiments automatically in your CI/CD pipelines.
- Package MLflow with your Docker image to ensure reproducible deployments.
- Combine with Kubernetes for scalable model serving.

Note

Rule of thumb: Use MLflow to track experiments and MLOps to operationalize your ML models.

8 Project Structure & Best Practices

Why Structure Matters

A well-structured project:

- Makes collaboration easier (everyone knows where things are).
- Simplifies debugging and scaling.
- Enables automation (CI/CD, testing, deployment).

Tip

Think of your project like a house: clear rooms, labeled cupboards, easy access.

—

General Layout

Here is a common folder structure for a data science or ML project:

```
my_project/

 Dockerfile
 docker-compose.yml
 requirements.txt   # or environment.yml
 notebooks/         # Jupyter or Colab notebooks
 src/               # Python source code (Flask app, ML scripts)
    __init__.py
    api/
    models/
 data/
    raw/           # unprocessed data
    processed/     # cleaned or feature data
    external/      # external datasets
 tests/             # unit/integration tests
 configs/           # YAML/JSON config files
 scripts/           # helper scripts (training, ETL, deployment)
 .gitignore
 README.md
```

—

Dockerfile Best Practices

Best Practices

- Always start from a lightweight base image (`python:3.10-slim`).
- Use `requirements.txt` or `poetry.lock` for dependencies.
- Separate layers: first install dependencies, then copy code.
- Keep the container stateless: never hardcode local paths.

Example:

```
FROM python:3.10-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY src/ .
CMD ["python", "app.py"]
```

—

Architecting End-to-End Projects

Tip

Rule of thumb: Notebooks = experiments. Src/ = production code.

Steps for structuring a real ML or data-driven project:

1. **Data**: organize raw vs processed datasets.
2. **Exploration**: notebooks for experiments.
3. **Source Code**: reusable modules in `src/`.
4. **APIs**: expose your model via Flask/FastAPI.
5. **Containerization**: Dockerfile + docker-compose.
6. **Testing**: add automated unit tests in `tests/`.
7. **CI/CD**: GitHub Actions pipeline for build & deploy.
8. **Monitoring**: Prometheus/Grafana for performance.

—

Hands-On Lab: Structuring Your Project

Lab Steps

1. Create a new folder `mlproject/`.
2. Inside, create `src/`, `data/`, and `notebooks/`.
3. Write a simple Flask API inside `src/app.py`.
4. Add a `Dockerfile` and test by building the image.
5. Push the project to GitHub.

Questions

- Why separate notebooks from production code?
- How does `docker-compose.yml` fit into project structure?
- Which parts of the project should never be pushed to GitHub (hint: raw data, secrets)?

—

Best Practices Checklist

Checklist

- Always use `.gitignore` for large data and secrets.
- Keep config in files (YAML/JSON), not hardcoded.
- Use virtual environments (conda/venv/poetry).
- Write unit tests before deploying.
- Document in `README.md`.